Part 3 Technical Reflection Report

GLMS Modernisation: Docker and Automated Testing

The migration of the Global Logistics Management System (GLMS) from a monolithic ASP.NET Core MVC application to a service-oriented architecture significantly improved separation of concerns, deployment flexibility, and quality assurance. In the Part 3 implementation, the system was split into a backend Web API, a frontend MVC client, and a SQL Server database, with each component running in isolated containers.

1. Why Automated Testing Is Critical in a CI/CD Pipeline

Automated testing is a core quality gate in modern enterprise delivery. In a CI/CD pipeline, code changes are merged frequently and deployed rapidly. Without automated tests, each release depends on manual verification, which is slow, inconsistent, and error-prone.

In this GLMS implementation, automated tests provide the following benefits:

- Fast feedback on regressions:
  API integration tests automatically call real endpoints and verify HTTP status codes and response payloads. For example, a test calls GET /api/contracts and asserts a 200 OK response with non-empty JSON.

- Confidence in refactoring:
  During migration from direct database access in MVC controllers to HttpClient-based API calls, tests ensured existing business behavior was preserved.

- Repeatability:
  Tests run the same way on any machine and in any pipeline stage, reducing human variability.

- Release safety:
  If a breaking change is introduced (for example, route mismatch, invalid auth setup, or contract response shape changes), tests fail before deployment.

- Better DevOps culture:
  Teams can treat quality checks as code, not as a manual final step.

In practical CI/CD terms, automated tests prevent bugs from reaching production by failing the build artifact early. A typical pipeline stage order is:

1. Restore and build.
2. Execute unit and integration tests.
3. Block deployment on failure.
4. Deploy only verified artifacts.

This means defects are caught when they are cheapest to fix, instead of after customer impact.

2. Containerization and Environment Consistency

One of the biggest risks in software delivery is environment drift, commonly described as: â€œit works on my machine.â€ Docker directly addresses this by packaging application code, runtime, dependencies, and configuration into immutable images.

In GLMS, Docker was used to containerize:

- sql-server-db: persistent SQL Server database
- glms-backend-api: ASP.NET Core Web API
- glms-frontend-web: ASP.NET Core MVC client

These services are orchestrated through docker-compose with a shared internal network. This provides several consistency and reliability advantages:

- Identical runtime behavior across Dev, Test, and Prod:
  The same container image runs in each environment, reducing hidden machine-specific differences.

- Explicit dependencies and startup order:
  docker-compose captures service relationships and health checks, such as API waiting on SQL Server readiness.

- Simplified onboarding:
  New developers can start the complete system with one command instead of manually configuring local SQL Server and runtime settings.

- Isolation:
  Each service runs independently, preventing local tool/version conflicts.

- Scalable architecture foundation:
  Because frontend and backend are decoupled, they can scale independently in future cloud deployments.

Containerization therefore improves reliability, repeatability, and deployment confidence. It also provides a direct path toward cloud-native hosting and automated release pipelines.

3. Architectural Reflection

The Part 3 refactor produced clear enterprise benefits:

- Separation of layers:
  MVC is now presentation-focused, while business logic and data access are centralized in the API.

- Contract-first integration:
  Frontend-to-backend communication is now explicit through HTTP endpoints and JSON contracts.

- Security hardening baseline:
  JWT authentication was introduced so API endpoints require bearer tokens, reducing unauthenticated data access.

- Testability:
  The API can be tested independently from the frontend UI, making automated quality checks easier and more reliable.

4. Conclusion

Automated testing and containerization are not optional enhancements for enterprise systems; they are core operational requirements. In GLMS, automated tests protect against regressions during rapid iteration, while Docker ensures consistent behavior across environments. Together, they improve software quality, reduce deployment risk, and support scalable DevOps delivery.

This modernized architecture positions GLMS for future enhancements such as horizontal scaling, API gateway integration, centralized observability, and cloud-based deployment strategies.